Dziedziczenie a obiekty wartości — prostokąt i kwadrat

Z matematyki wiemy, że każdy kwadrat jest prostokątem. Czy klasa `Kwadrat` powinna wobec tego dziedziczyć po klasie `Prostokat`? Odpowiedź zależy również od operacji, które udostępniają te klasy.

To materiał dodatkowy do dziedziczenia. Wracamy w nim do rozróżnienia obiektów wartości i obiektów, których tożsamość ma znaczenie. Będziemy stopniowo zmieniać model i sprawdzać, jakie obietnice spełniają jego operacje.

Prostokąt o zmiennych wymiarach

Pierwsza wersja pozwala zmieniać osobno szerokość i wysokość. Przyjmijmy umowę: `zmien_wysokosc` zmienia wysokość i pozostawia szerokość bez zmian; `zmien_szerokosc` działa analogicznie.

W tej prostej wersji pola są publiczne, więc można również przypisywać im wartości bezpośrednio. Problem, który zaraz zobaczymy, występuje nawet wtedy, gdy program korzysta wyłącznie z metod zmieniających wymiary.

In [1]:
```
class Prostokat:
    def __init__(self, szerokosc, wysokosc):
        self.szerokosc = szerokosc
        self.wysokosc = wysokosc

    def zmien_szerokosc(self, nowa_szerokosc):
        self.szerokosc = nowa_szerokosc

    def zmien_wysokosc(self, nowa_wysokosc):
        self.wysokosc = nowa_wysokosc

    def pole(self):
        return self.szerokosc * self.wysokosc

    def __repr__(self):
        return f"Prostokat({self.szerokosc}x{self.wysokosc})"
```

In [2]:
```
prostokat = Prostokat(5, 2)
```

In [3]:
```
prostokat.zmien_wysokosc(10)
```

In [4]:
```
prostokat
```

Out[4]:
```
Prostokat(5x10)
```

In [5]:
```
prostokat.pole()
```

Out[5]:
```
50
```

Pierwsza próba: kwadrat dziedziczy wszystkie operacje

W konstruktorze kwadratu ustawimy obydwa wymiary na tę samą wartość. Dodamy też wygodną metodę zmieniającą długość boku.

In [6]:
```
class NaiwnyKwadrat(Prostokat):
    def __init__(self, bok):
        super().__init__(bok, bok)

    def zmien_dlugosc_boku(self, nowy_bok):
        self.szerokosc = nowy_bok
        self.wysokosc = nowy_bok

    def __repr__(self):
        return f"NaiwnyKwadrat({self.szerokosc}x{self.wysokosc})"
```

In [7]:
```
kwadrat = NaiwnyKwadrat(5)
```

In [8]:
```
kwadrat
```

Out[8]:
```
NaiwnyKwadrat(5x5)
```

In [9]:
```
kwadrat.zmien_dlugosc_boku(7)
```

In [10]:
```
kwadrat
```

Out[10]:
```
NaiwnyKwadrat(7x7)
```

Ta operacja zachowała równość boków. Obiekt odziedziczył jednak także pozostałe metody `Prostokat`.

In [11]:
```
kwadrat.zmien_wysokosc(10)
```

In [12]:
```
kwadrat
```

Out[12]:
```
NaiwnyKwadrat(7x10)
```

Otrzymaliśmy `NaiwnyKwadrat(7x10)`. Klasa ma w nazwie kwadrat, ale jej obiekt nie zachowuje podstawowej własności kwadratu: równości boków. Dziedziczenie metod może udostępnić operacje, które nie pasują do modelu klasy pochodnej.

Druga próba: kwadrat pilnuje równości boków

Nadpiszemy obie odziedziczone metody. Każda z nich zmieni teraz obydwa wymiary.

In [13]:
```
class Kwadrat(Prostokat):
    def __init__(self, bok):
        super().__init__(bok, bok)

    def zmien_szerokosc(self, nowy_bok):
        self.szerokosc = nowy_bok
        self.wysokosc = nowy_bok

    def zmien_wysokosc(self, nowy_bok):
        self.szerokosc = nowy_bok
        self.wysokosc = nowy_bok

    def __repr__(self):
        return f"Kwadrat({self.szerokosc}x{self.wysokosc})"
```

In [14]:
```
kwadrat = Kwadrat(5)
```

In [15]:
```
kwadrat.zmien_wysokosc(10)
```

In [16]:
```
kwadrat
```

Out[16]:
```
Kwadrat(10x10)
```

Boki nadal są równe. Czy taka klasa może zastępować prostokąt w programie, który korzysta z wcześniejszej umowy?

Program korzystający z prostokąta

Wyobraźmy sobie zmianę wysokości obrazu. Program zna aktualną szerokość i zakłada, że zmiana wysokości jej nie zmieni. Po ustawieniu wysokości na `10` pole powinno wynieść `szerokosc_przed * 10`.

Ważne jest zapisanie szerokości **przed** operacją. Gdybyśmy odczytali ją dopiero po zmianie, sprawdzenie nie wykryłoby naruszenia tej obietnicy.

In [17]:
```
def przetestuj_prostokat(prostokat):
    print(f"Otrzymałem: {prostokat}")
    szerokosc_przed = prostokat.szerokosc
    prostokat.zmien_wysokosc(10)
    oczekiwane_pole = szerokosc_przed * 10
    rzeczywiste_pole = prostokat.pole()
    print(f"Po zmianie wysokości na 10: {prostokat}")
    if rzeczywiste_pole == oczekiwane_pole:
        print("Wszystko działa zgodnie z oczekiwaniami!")
    else:
        print(
            f"BŁĄD! Oczekiwano pola {oczekiwane_pole}, "
```

```
            f"otrzymano {rzeczywiste_pole}"
        )
```

In [18]:
```
zwykly_prostokat = Prostokat(5, 2)
```

In [19]:
```
przetestuj_prostokat(zwykly_prostokat)
```

```
Otrzymałem: Prostokat(5x2)
Po zmianie wysokości na 10: Prostokat(5x10)
Wszystko działa zgodnie z oczekiwaniami!
```

In [20]:
```
moj_kwadrat = Kwadrat(5)
```

In [21]:
```
przetestuj_prostokat(moj_kwadrat)
```

```
Otrzymałem: Kwadrat(5x5)
Po zmianie wysokości na 10: Kwadrat(10x10)
BŁĄD! Oczekiwano pola 50, otrzymano 100
```

Dla kwadratu otrzymaliśmy pole `100`, chociaż program oczekiwał `50`. Nie wystąpił wyjątek. Problem polega na tym, że `Kwadrat.zmien_wysokosc` zmienia również szerokość, wbrew umowie klasy bazowej.

Co wynika z tego dla dziedziczenia?

To przykład związany z **zasadą podstawienia Liskov**: kod korzystający z umowy klasy bazowej powinien zachowywać poprawność również wtedy, gdy dostanie obiekt klasy pochodnej.

Samo nadpisanie metody jest zwykłym mechanizmem dziedziczenia. Tutaj trudność wynika ze zmiany jej znaczenia. Liczba nadpisanych metod także nie rozstrzyga, czy projekt jest dobry — trzeba sprawdzić, czy zachowują umowę.

Nie wynika z tego zakaz tworzenia mutowalnych figur. W edytorze grafiki możemy potrzebować obiektu, który zachowuje tożsamość przy zmianie rozmiaru. Możemy wtedy zdefiniować prostokąt i kwadrat jako osobne klasy, korzystające ze wspólnego interfejsu, np. metody `pole`.

Inna umowa: obiekt opisuje wartość

Możemy potraktować wymiary figury jako **wartość**. W takim modelu prostokąt o wymiarach `5 x 2` i prostokąt `5 x 10` reprezentują różne wartości. Operacja może obliczać nową figurę, pozostawiając wcześniejszy obiekt bez zmian.

Nie nazywamy jej teraz `zmien_wysokosc`, lecz `z_wysokoscia`: wynikiem będzie nowy prostokąt o podanej wysokości. Nie wymagamy, żeby wynik miał tę samą klasę co argument. Rozciągnięcie kwadratu może dać zwykły prostokąt.

Użyjemy znanych już właściwości tylko do odczytu. Zwykły kod korzystający z publicznego interfejsu odczytuje wymiary, ale nie przypisuje im nowych wartości. Pola z podkreśleniem pozostają szczegółem implementacji, którego nie należy zmieniać z zewnątrz.

In [22]:
```
class ProstokatV0:
    def __init__(self, szerokosc, wysokosc):
        if szerokosc <= 0 or wysokosc <= 0:
            raise ValueError("Wymiary muszą być dodatnie.")
        self._szerokosc = szerokosc
        self._wysokosc = wysokosc

    @property
    def szerokosc(self):
        return self._szerokosc

    @property
    def wysokosc(self):
        return self._wysokosc

    def z_szerokoscia(self, nowa):
        return ProstokatV0(nowa, self.wysokosc)

    def z_wysokoscia(self, nowa):
        return ProstokatV0(self.szerokosc, nowa)

    def pole(self):
        return self.szerokosc * self.wysokosc

    def __eq__(self, inny):
        if not isinstance(inny, ProstokatV0):
            return NotImplemented
        return (
            self.szerokosc == inny.szerokosc
            and self.wysokosc == inny.wysokosc
        )

    def __repr__(self):
        return f"ProstokatV0({self.szerokosc}x{self.wysokosc})"
```

`KwadratV0` ustawia dwa równe wymiary. Nie musi nadpisywać metod tworzących wynik o innych wymiarach: odziedziczone metody obiecują zwrócić prostokąt i właśnie to robią.

In [23]:
```
class KwadratV0(ProstokatV0):
    def __init__(self, bok):
        super().__init__(bok, bok)

    def __repr__(self):
        return f"KwadratV0({self.szerokosc}x{self.wysokosc})"
```

In [24]:
```
moj_matematyczny_kwadrat = KwadratV0(5)
```

In [25]:
```
moj_matematyczny_kwadrat
```

Out[25]:
```
KwadratV0(5x5)
```

In [26]:
```
nowa_figura = moj_matematyczny_kwadrat.z_wysokoscia(10)
```

In [27]:
```
nowa_figura
```

Out[27]:
```
ProstokatV0(5x10)
```

In [28]:
```
moj_matematyczny_kwadrat
```

Out[28]:
```
KwadratV0(5x5)
```

In [29]:
```
moj_matematyczny_kwadrat.pole(), nowa_figura.pole()
```

Out[29]:
```
(25, 50)
```

In [30]:
```
type(nowa_figura), type(moj_matematyczny_kwadrat)
```

Out[30]:
```
(__main__.ProstokatV0, __main__.KwadratV0)
```

Stary kwadrat ma nadal boki długości `5`. Wynikiem operacji jest prostokąt `5 x 10`. Przypisanie `moj_matematyczny_kwadrat.wysokosc = 10` zgłosi `AttributeError`, ponieważ właściwość nie ma metody ustawiającej.

In [31]:
```
moj_matematyczny_kwadrat.wysokosc = 10
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[31], line 1
----> 1 moj_matematyczny_kwadrat.wysokosc = 10

AttributeError: property 'wysokosc' of 'KwadratV0' object has no setter
```

Wartość a tożsamość

Dwa obiekty mogą reprezentować tę samą wartość, mimo że są odrębnymi obiektami Pythona. W naszym modelu o wartości decydują dwie długości boków; nie przechowujemy położenia ani innych cech figury.

In [32]:
```
pierwsza = KwadratV0(5)
druga = ProstokatV0(5, 5)
```

In [33]:
```
pierwsza == druga
```

Out[33]:
```
True
```

In [34]:
```
pierwsza is druga
```

```
Out[34]: False
```

Każdy obiekt Pythona ma tożsamość, także niemutowalny. W obiekcie wartości zwykle nie jest ona tym, co chcemy porównywać. Niemutowalność ułatwia bezpieczne współdzielenie wartości, ale nie jest definicją braku tożsamości.

Wcześniejszy przykład tras pokazuje też, że decyzja o traktowaniu czegoś jako wartości nie narzuca jednej techniki implementacji w każdym programie. Tu wybraliśmy brak publicznych operacji zmieniających stan, aby jasno określić znaczenie operacji i zachować wcześniejsze wartości.

O czym trzeba zdecydować?

Dziedziczenie powinno odpowiadać umowie używanej przez program. W szczególności trzeba ustalić:

- czy operacja zmienia argument, czy tworzy nowy obiekt;
- jakie własności wcześniejszego obiektu pozostają zachowane;
- jakie typy wyników są dopuszczalne.

Gdyby `z_wysokoscia` miała obowiązek zwracać obiekt tej samej klasy co oryginał, nasza implementacja znów nie spełniałaby tej umowy dla kwadratu. Samo wprowadzenie niemutowalności nie rozwiązuje każdej trudności z dziedziczeniem.

Ćwiczenia sprawdzające

1. Dlaczego `isinstance(moj_kwadrat, Prostokat)` zwraca `True`, chociaż funkcja `przetestuj_prostokat` wykryła problem?
2. Co stałoby się ze sprawdzeniem, gdyby oczekiwane pole obliczało na podstawie szerokości odczytanej po zmianie wysokości? Sprawdź to w osobnej wersji funkcji.
3. Czy usunięcie metod `zmien_szerokosc` i `zmien_wysokosc` z pierwszej klasy wystarczyłoby do uniemożliwienia zmiany jej stanu?
4. Utwórz `KwadratV0(5)` i wywołaj dla niego `z_szerokoscia(8)`. Sprawdź wymiary i typ wyniku oraz wymiary wcześniejszego obiektu.
5. Zastanów się, czy w tym modelu `KwadratV0(5)` i `ProstokatV0(5, 5)` powinny porównywać się jako równe. Która metoda o tym decyduje? Czy muszą być tym samym obiektem?

Przykładowe odpowiedzi

Dokumentacja: klasy i dziedziczenie, `property`.